

Finding All Approximate Palindromes

L. C. Chen and R. C. T. Lee

Department of Computer Science and Information Engineering,

National Chi Nan University

{s95321503,rctlee}@ncnu.edu.tw

Abstract

In this paper, we discuss the approximate palindrome finding problem. An approximate palindrome with error k is a string $S = S_1S_2$ or $S = S_1aS_2$ such that the edit distance between S_1 and the reverse of S_2 is smaller than or equal to k . We give an algorithm to solve the all approximate palindrome finding problem defined as follows: Given a text string T and a k , find all occurrences of approximate palindromes with error k' in T for $0 \leq k' \leq k$.

1. Introduction

A string S is a palindrome if $S = \bar{S}$ where $\bar{S}$ denotes the reverse of S . The palindrome finding problem is defined as follows: Given a string S , find all palindromes within S . Algorithms to solve this problem can be found in [9].

We now give the palindrome a new definition: A string S is a palindrome if it is of the form of S_1S_2 or S_1aS_2 such that $\bar{S}_1$ exactly matches with S_2 . A palindrome is even(odd) if it is in the form of S_1S_2 (S_1aS_2).

Based upon the above definitions, we define an approximate palindrome with error k as a string of the form of S_1S_2 or S_1aS_2 such that $ED(\bar{S}_1, S_2) \leq k$ where $ED(A, B)$ denotes the edit distance between two strings A and B . It is customary to define the edit distance between two strings A and B , denote $ED(A, B)$, as the minimum number of substitutions, insertions and deletions of characters in A to transform A to B . Edit distances can be found by using the dynamic programming approach [3, 8 and 14]. In [13], the maximal approximate palindrome with error k was defined and algorithms to solve the maximal approximate palindrome finding problem was given.

In this paper, we shall solve the all approximate palindromes with error k finding problem.

2. The All Approximate Palindrome Finding Problem

We define the all approximate palindrome finding problem as follows: Given a text T and a k , find all occurrences of approximate palindromes with error at most k' , $0 \leq k' \leq k$, in T . Before we present the algorithm to solve the problem, let us note an important point here, we call it "length property". Suppose we have strings A and B , and $|A| = m$. If $ED(A, B) \leq k$, then $m - k \leq |B| \leq m + k$. That is, the length of B can be neither too small nor too large. This property has been used in many papers [2, 5, 10, 11, and 15]. We now use the above property to solve the approximate palindrome finding problem. In the following, we let $T = t_1t_2 \cdots t_n$ and let $T = (i, j)$ denote $t_it_{i+1} \cdots t_j$.

Before giving the details of our algorithm, let us assume that we are going to find all even approximate palindromes. We perform a linear scan of the text T . To further simplify our discussion, we assume that $i \leq \lfloor n/2 \rfloor$. Let S_1 denote $\overline{T(1, i)}$ and S_2 denote $T(i+1, \min\{2i+k, n\})$. Our problem now becomes the following problem: Given a k' , $k' = 0, 1, \dots, k$ find each prefix A of S_2 such that $ED(S_1(1, i'), A) = k'$, where $1 \leq i' \leq i$

Thus our problem is the same as the approximate string matching problem defined as follows: Given a text $T = t_1t_2 \cdots t_n$, a pattern $P = p_1p_2 \cdots p_m$ and a error bound k , find all the substrings of T whose edit distances with P are less than or equal to k . Many researches

focus on algorithms to find the edit distance between two strings, and many of them [1, 3, 4, 6, 7, 10, 11, 12, 14, 15, and 16] use the following rule: Consider the two strings $A = A_1A_2$ and $B = B_1B_2$, if $A_2 = B_2$ and $ED(A_1, B_1) \leq k$, then $ED(A, B) \leq k$.

There are two approaches to solve the approximate string matching problem. The first approach [12, 2, 10, 11 and 15] is a local approach which computes $ED(P, S)$ where S is a substring of T starting from each location i . That is, for every possible position i , we test whether there exists a substring S of T starting from this location such that $ED(P, S) \leq k$. Another approach [16], named Approach 2, is a global approach which can solve the problem in $O(nk \lceil \frac{m}{w} \rceil)$ time-complexity where w is the word size. We shall use the technique in [16] to solve the problem.

In the above, we only talked about one case. In the following, we give the precise definition of S_1 and S_2 for all cases.

Case 1: We find even approximate palindromes. If $1 \leq i \leq \lfloor n/2 \rfloor$, let S_1 be $\overline{T(1, i)}$ and S_2 be $T(i+1, \min\{2i+k, n\})$; otherwise, let S_1 be $\overline{T(\max\{2i-n-k+1, 1\}, i)}$ and S_2 be $T(i+2, n)$.

Case 2: We find odd approximate palindromes.

If $1 \leq i \leq \lfloor n/2 \rfloor$, let S_1 be $\overline{T(1, i)}$ and S_2 be $T(i+2, \min\{2i+k+1, n\})$; otherwise, let S_1 be $\overline{T(\max\{2i-n-k+2, 1\}, i)}$ and S_2 be $T(i+2, n)$.

Essentially, for each location i in T , we compute the edit distance between S_1 and S_2 . Given two strings $S_1 = p_1p_2 \cdots p_m$, $S_2 = t_1t_2 \cdots t_{n'}$ and k , we define

$$M^k(i, j) = \begin{cases} 1, & \text{if } ED(S_1(1, j), S_2(1, i)) \leq k \\ 0, & \text{otherwise.} \end{cases},$$

where $0 \leq i \leq n'$ and $0 \leq j \leq m'$.

We now use dynamic approach to find $M^k(i, j)$.

Let $M_I^k(i, j)$, $M_D^k(i, j)$ and $M_S^k(i, j)$ denote the $M^k(i, j)$ related to insertion, deletion and substitution respectively, where

$$M_I^k(i, j) = \begin{cases} = 1, & \text{if } t_i \neq p_j, M^{k-1}(i-1, j) = 1 \\ & \text{or } t_i = p_j \text{ and } M^k(i-1, j-1) = 1 \\ = 0, & \text{otherwise.} \end{cases}$$

$$M_D^k(i, j) = \begin{cases} = 1, & \text{if } t_i \neq p_j, M^{k-1}(i, j-1) = 1 \\ & \text{or } t_i = p_j \text{ and } M^k(i-1, j-1) = 1 \\ = 0, & \text{otherwise.} \end{cases}$$

and

$$M_S^k(i, j) = \begin{cases} = 1, & \text{if } t_i \neq p_j \text{ and } M^{k-1}(i-1, j-1) = 1 \\ & \text{or } t_i = p_j \text{ and } M^k(i-1, j-1) = 1 \\ = 0, & \text{otherwise.} \end{cases}$$

After every $M_I^k(i, j)$, $M_D^k(i, j)$ and $M_S^k(i, j)$ are found, we can determine $M^k(i, j)$ by $M^k(i, j) = M_I^k(i, j)$ or $M_D^k(i, j)$ or $M_S^k(i, j)$ where “or” is the “logical or” operation.

Our algorithm is presented in the following:

Input: A string $T = t_1t_2 \cdots t_n$ and k .

Output: All occurrences of approximate palindromes of T with error k' .

```

for i=1 to n do
    for 0 ≤ k' ≤ k do
        for 0 ≤ i' ≤ n' and 0 ≤ j' ≤ m' do
             $M^{k'}(i', j') = \begin{cases} M^{k'}(0, j') = 1, & \text{if } 0 \leq j' \leq k' \\ M^{k'}(i', 0) = 1, & \text{if } 0 \leq i' \leq k' \\ 0 & \text{otherwise} \end{cases}$ 
             $M^{k'}(i', j') = M_I^{k'}(i', j') \text{ or } M_D^{k'}(i', j') \text{ or } M_S^{k'}(i', j')$ 
        end for
    end for
end for
    
```

An Example

We are given a string $T = abaaaacaaca$, $k=1$. Since there will be too many approximate palindromes if we consider every location i , we only present the result for $i=4$ and we only find the even approximate palindromes. In this case, $T(1,4) = abaa$, $S_1 = \overline{T(1,4)} = aaba$ and $S_2 = T(5,10) = aacaa$.

M^0

		a	a	c	a	a
	1	0	0	0	0	0
a	0	1	0	0	0	0
a	0	0	1	0	0	0
b	0	0	0	0	0	0
a	0	0	0	0	0	0

From the above table M^0 , aa and aaaa are all of the even approximate palindromes with error 0.

M^1

		a	a	c	a	a
	1	1	0	0	0	0
a	1	1	1	0	0	0
a	0	1	1	1	0	0
b	0	0	1	1	0	0
a	0	0	0	0	1	0

Again, we can see that a, aa, aaa, aaaa,baaaa, aaaac, baaaac and abaaaaca are all of the even approximate palindromes with error=1.

3. Concluding remarks

In this paper, we introduced the definition of approximate palindromes and the all approximate palindromes finding problem. Then we use the "length property" and the approach in [WM92] to solve this problem faster. In the future, we plan

to investigate the all approximate palindrome subsequence problem.

References

- [1] Amir, A., Keselman, D., Landau, G. M., Lewenstein, M., Lewenstein, N. and Rodeh, Text Indexing and Dictionary Matching with One Error, *M., Journal of Algorithms*, Vol. 37, 2000, pp. 309-325.
- [2] Fredriksson, K. and Navarro, G., Average-Optimal Multiple Approximate String Matching, *ACM Journal of Experimental Algorithmics*, Vol 9, No. 1.4, 2004, pp. 1-47.
- [3] Heikki H., Bit-parallel approximate string matching algorithms with transposition, *Journal of Discrete Algorithms*, Vol. 3, 2005, pp. 215-229.
- [4] Huynh, T. N. D., Hon, W. K., Lam, T. W. and Sung, W. K., Approximate String Matching Using Compressed Suffix Arrays, *Theoretical Computer Science*, Vol. 352, 2006, pp. 240-249.
- [5] Holub, J. and Melichar, B., Approximate string matching using factor automata, *Theoretical Computer Science*, Vol. 249, 2000, pp. 305-311.
- [6] Hyyro, H. and Navarro, G., Bit-parallel Witnesses and their Applications to Approximate String Matching, *Algorithmica*, Vol 4, No. 3, 2005, pp. 203-231.
- [7] Landau, G. and Vishkin, U., Fast parallel and serial approximate string matching, *Journal of algorithms*, Vol. 10, 1989, pp. 157-169.
- [8] Lee, R.C.T, Tseng, S. S., Chang, R. C. and Chang, Y. T., Introduction to the design and analysis of algorithms, *Mc Graw Hill*, 2005, pp. 253-316.
- [9] Manacher, G., A new linear-time on-line algorithm for finding the smallest initial palindrome of the string, *J. ACM* 22, 1975, pp. 8-19.
- [10] Navarro, G. and Baeza-Yates, R., Very fast and simple approximate string matching, *Information Processing Letters*, Vol. 72, 1999, pp. 65-70.
- [11] Navarro, G. and Baeza-Yates, R., A Hybrid Indexing Method for Approximate String Matching, *J. Discret. Algorithms*, Vol.1, No.1, 2000, pp. 205-239.
- [12] Needleman, S. B. and Wunsch, C. D., A general method applicable to the search for similarities in the aminoacid sequence of two proteins, *Journal of Molecular Biology*, Vol. 48, 1970, pp.

- 443-453.
- [13] Porto, A. H. L and Barbosa, V. C., Finding approximate palindromes in strings, *Pattern Recognition*, Vol.35, 2002, pp. 2581-2591.
 - [14] Sellers, P. H., String Matching with Errors, *Journal of Algorithms*, Vol. 20, No. 1, 1980, pp. 359-373.
 - [15] Tarhio, J. and Ukkonen, E., Approximate Boyer-Moore String Matching, *SIAM Journal on Computing*, Vol. 22, No. 2, 1993, pp. 243-260.
 - [16] Wu, S. and Manber, U., Fast Text Searching: Allowing Errors, *Communications of the ACM*, Vol. 35, 1992, pp. 83-91.